Evan Witte
Kimberly Wang
Ben Goldberg

Studio A

1. VHDL code

PARITY EVEN/ODD

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity even_odd is
        Port ( a, b : in std_logic_vector(3 downto 0);
        same : out STD_LOGIC);
end even_odd;

architecture Behavioral of even_odd is
        signal parity_1: STD_LOGIC;
        signal parity_2: STD_LOGIC;
begin
        parity_1 <= a(3) xor a(2) xor a(1) xor a(0);
        parity_2 <= b(3) xor b(2) xor b(1) xor b(0);
        same <= parity_1 xnor parity_2;
end Behavioral;
```

ADDER

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Full_Adder is
        port(a, b, c_in : in std_logic;
   s, c_out : out std_logic);
end Full_Adder;

architecture Behavioral of Full_Adder is
begin
        s <= a xor b xor c_in;
        c_out <= ( (a xor b) and c_in ) or (a and b);
end Behavioral;

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity fourBitAdder is
   Port ( a : in  STD_LOGIC_VECTOR (3 downto 0);
        b : in  STD_LOGIC_VECTOR (3 downto 0);
        cin : in  STD_LOGIC;
        s : out  STD_LOGIC_VECTOR (3 downto 0);
        cout : out  STD_LOGIC);
end fourBitAdder;

architecture Behavioral of fourBitAdder is

        component Full_Adder is
                port (a, b, c_in: in STD_LOGIC;
                        s, c_out: out STD_LOGIC);
        end component;
        signal c: STD_LOGIC_VECTOR(4 downto 0);
begin
        u1: Full_Adder port map (a => a(0), b => b(0), c_in => c(0), s => s(0), c_out => c(1));
        u2: Full_Adder port map (a => a(1), b => b(1), c_in => c(1), s => s(1), c_out => c(2));
        u3: Full_Adder port map (a => a(2), b => b(2), c_in => c(2), s => s(2), c_out => c(3));
        u4: Full_Adder port map (a => a(3), b => b(3), c_in => c(3), s => s(3), c_out => c(4));
        c(0) <= cin;
        cout <= c(4);

end Behavioral;
```

Subtractor Files

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Full_Sub is
        port(a, b, c_in : in std_logic;
   d, borrow : out std_logic);
end Full_Sub;

architecture data of full_sub is
begin
  d <= a xor b xor c_in;
  borrow <= ((b xor c_in) and (not a)) or (b and c_in);
end data;

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity fourBitSubtr is
   Port ( a : in  STD_LOGIC_VECTOR (3 downto 0);
        b : in  STD_LOGIC_VECTOR (3 downto 0);
        cin : in  STD_LOGIC;
        d : out  STD_LOGIC_VECTOR (3 downto 0);
        cout : out  STD_LOGIC);
end fourBitSubtr;

architecture Behavioral of fourBitSubtr is

        component Full_Sub is
                port (a, b, c_in: in STD_LOGIC;
                        d, borrow: out STD_LOGIC);
        end component;
        signal c: STD_LOGIC_VECTOR(4 downto 0);
        signal conv: STD_LOGIC_VECTOR(4 downto 0);
        signal twoscomp: STD_LOGIC_VECTOR(3 downto 0);
        signal onescomp: STD_LOGIC_VECTOR(3 downto 0);

begin
   c(0) <= cin;
        u1: Full_Sub port map (a => a(0), b => b(0), c_in => c(0), d => twoscomp(0), borrow =>
c(1));
        u2: Full_Sub port map (a => a(1), b => b(1), c_in => c(1), d => twoscomp(1), borrow =>
c(2));
```

```
        u3: Full_Sub port map (a => a(2), b => b(2), c_in => c(2), d => twoscomp(2), borrow =>
c(3));
        u4: Full_Sub port map (a => a(3), b => b(3), c_in => c(3), d => twoscomp(3), borrow =>
c(4));
        cout <= c(4);

        c1: Full_Sub port map (a => twoscomp(0), b => c(4), c_in => conv(0), d =>
onescomp(0), borrow => conv(1));
        c2: Full_Sub port map (a => twoscomp(1), b => '0', c_in => conv(1), d => onescomp(1),
borrow => conv(2));
        c3: Full_Sub port map (a => twoscomp(2), b => '0', c_in => conv(2), d => onescomp(2),
borrow => conv(3));
        c4: Full_Sub port map (a => twoscomp(3), b => '0', c_in => conv(3), d => onescomp(3),
borrow => conv(4));

        d(0) <= onescomp(0) XOR c(4);
        d(1) <= onescomp(1) XOR c(4);
        d(2) <= onescomp(2) XOR c(4);
        d(3) <= onescomp(3) XOR c(4);
        d(3) <= onescomp(3) XOR c(4);

end Behavioral;
```

Comparator:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity comparator is
        Port ( a, b : in std_logic_vector(3 downto 0);
        greater : out STD_LOGIC;
        equal : out STD_LOGIC;
        less : out STD_LOGIC);
end comparator;

architecture Behavioral of comparator is

begin
  process (a, b)
    begin
    if (a > b) then
       greater <= '1';
       less <= '0';
       equal <= '0';
    elsif (b > a) then
       greater <= '0';
       less <= '1';
       equal <= '0';
    else
       greater <= '0';
       less <= '0';
       equal <= '1';
    end if;
  end process;

end Behavioral;
```

Top Level

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
entity main is
    Port ( int1, int2 : in STD_LOGIC_VECTOR(3 downto 0);
           func : in STD_LOGIC_VECTOR(1 downto 0);
           LED0, LED1, LED2 : out STD_LOGIC;
           segments : out STD_LOGIC_VECTOR(6 downto 0);
           an : out  STD_LOGIC_VECTOR(0 to 3);
           clk : in STD_LOGIC);

end main;

architecture Behavioral of main is
    component fourBitAdder is
                    Port ( a, b : in  STD_LOGIC_VECTOR (3 downto 0);
          cin : in  STD_LOGIC;
          s : out  STD_LOGIC_VECTOR (3 downto 0);
          cout : out  STD_LOGIC);
    end component;

    component fourBitSubtr is
    Port ( a, b : in  STD_LOGIC_VECTOR (3 downto 0);
          cin : in  STD_LOGIC;
          d : out  STD_LOGIC_VECTOR (3 downto 0);
          cout : out  STD_LOGIC);
    end component;

    component comparator is
           Port ( a, b : in std_logic_vector(3 downto 0);
           greater, equal, less: out STD_LOGIC);
    end component;

    component even_odd is
           Port ( a, b : in std_logic_vector(3 downto 0);
           same : out STD_LOGIC);
    end component;

    component LEDdisplay IS
           Port (
clk : in  STD_LOGIC;
                    seg0,seg1,seg2,seg3 : in  STD_LOGIC_VECTOR (6 downto 0);
          seg : out  STD_LOGIC_VECTOR (6 downto 0);
                    a: out STD_LOGIC_VECTOR (3 downto 0));
```

```
    end component;

    component display_driver is
    Port ( inputs : in  STD_LOGIC_VECTOR (3 downto 0);
           seg_out : out  STD_LOGIC_VECTOR (6 downto 0));
    end component;

    signal val0, val1 : STD_LOGIC_VECTOR(3 downto 0);
    signal segment0, segment1 : STD_LOGIC_VECTOR(6 downto 0);

    signal val_add, val_sub: STD_LOGIC_VECTOR(3 downto 0);
    signal carry_add, led_sub: STD_LOGIC;
    signal led_gr, led_eq, led_le: STD_LOGIC;
    signal led_same: STD_LOGIC;
begin

display : LEDdisplay port map (clk => clk,
 seg0 => "1111111", seg1 => "1111111", seg2 => segment1, seg3 => segment0, seg =>
segments, a => an);
digit0 : display_driver port map (inputs => val0, seg_out => segment0);
digit1 : display_driver port map (inputs => val1, seg_out => segment1);

adder : fourBitAdder port map (a => int1, b => int2, cin => '0', s => val_add, cout => carry_add);
subtr : fourBitSubtr port map (a => int1, b => int2, cin => '0', d => val_sub, cout => led_sub);
comp : comparator port map (a => int1, b => int2, greater => led_gr, equal => led_eq, less =>
led_le);
parity : even_odd port map (a => int1, b => int2, same => led_same);

    change_mode : process(func,int1,int2)
    begin
        if func = "00" then
            val0 <= val_add;
            LED0 <= '0';
            LED1 <= '0';
            LED2 <= '0';
            if carry_add = '0' then
                val1 <= "0000";
            else
                val1 <= "0001";
            end if;
        elsif func = "01" then
            val0 <= val_sub;
            val1 <= "0000";
            LED0 <= '0';
```

```
            LED1 <= '0';
            LED2 <= led_sub;
        elsif func = "10" then
            val0 <= "0000";
            val1 <= "0000";
            LED0 <= led_le;
            LED1 <= led_eq;
            LED2 <= led_gr;
        elsif func = "11" then
            val0 <= "0000";
            val1 <= "0000";
            LED0 <= led_same;
            LED1 <= '0';
            LED2 <= '0';
        end if;
    end process change_mode;

end Behavioral;
```

2. XDC file

```
## 7 segment display
set_property PACKAGE_PIN W7 [get_ports {segments[0]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {segments[0]}]
set_property PACKAGE_PIN W6 [get_ports {segments[1]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {segments[1]}]
set_property PACKAGE_PIN U8 [get_ports {segments[2]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {segments[2]}]
set_property PACKAGE_PIN V8 [get_ports {segments[3]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {segments[3]}]
set_property PACKAGE_PIN U5 [get_ports {segments[4]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {segments[4]}]
set_property PACKAGE_PIN V5 [get_ports {segments[5]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {segments[5]}]
set_property PACKAGE_PIN U7 [get_ports {segments[6]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {segments[6]}]
set_property PACKAGE_PIN U2 [get_ports {an[0]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {an[0]}]
set_property PACKAGE_PIN U4 [get_ports {an[1]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {an[1]}]
set_property PACKAGE_PIN V4 [get_ports {an[2]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {an[2]}]
set_property PACKAGE_PIN W4 [get_ports {an[3]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {an[3]}]

## LEDs
set_property PACKAGE_PIN U19 [get_ports {LED0}]
   set_property IOSTANDARD LVCMOS33 [get_ports {LED0}]
set_property PACKAGE_PIN E19 [get_ports {LED1}]
   set_property IOSTANDARD LVCMOS33 [get_ports {LED1}]
set_property PACKAGE_PIN U16 [get_ports {LED2}]
   set_property IOSTANDARD LVCMOS33 [get_ports {LED2}]

## inputs
set_property PACKAGE_PIN V17 [get_ports {int1[0]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {int1[0]}]
set_property PACKAGE_PIN V16 [get_ports {int1[1]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {int1[1]}]
set_property PACKAGE_PIN W16 [get_ports {int1[2]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {int1[2]}]
set_property PACKAGE_PIN W17 [get_ports {int1[3]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {int1[3]}]
```

```
set_property PACKAGE_PIN W15 [get_ports {int2[0]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {int2[0]}]
set_property PACKAGE_PIN V15 [get_ports {int2[1]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {int2[1]}]
set_property PACKAGE_PIN W14 [get_ports {int2[2]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {int2[2]}]
set_property PACKAGE_PIN W13 [get_ports {int2[3]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {int2[3]}]

set_property PACKAGE_PIN T1 [get_ports {func[0]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {func[0]}]
set_property PACKAGE_PIN R2 [get_ports {func[1]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {func[1]}]

set_property PACKAGE_PIN W5 [get_ports clk]
        set_property IOSTANDARD LVCMOS33 [get_ports clk]
        create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports clk]

set_property CLOCK_DEDICATED_ROUTE FALSE [get_nets func_IBUF[1]]
```

3). Boolean Expression Tables

Adder

Input 1	Input 2	Expected Binary Output	Expected Hex Output	Actual Hex Output
1111	0000	1111	0F	0F
1010	1100	00010110	16	16
1100	0110	00010010	12	12
0011	1110	00010001	11	11

Subtractor

Input 1	Input 2	Expected Binary Output	Expected Hex Output	Actual Hex Output
1111	0000	1111	0F	0F
1010	1001	0001	01	01
1100	0110	0110	06	06
0011	0001	0010	04	04

Comparator: (LED 2 is input 1 bigger, LED 1 is equal, and LED 0 is input 2 is bigger)

Input 1	Input 2	Expected LED Output	Actual LED Output
1111	0000	LED 2	LED 2
1010	1100	LED 0	LED0
1100	1100	LED 1	LED1
0011	1110	LED 0	LED0

Parity: (LED 2 is on if inputs are equal parity, off when they aren't),

Input 1	Input 2	Expected LED 2 Output	Actual LED 2 Output
1111	0000	ON	ON
1011	1100	OFF	OFF
1100	0110	ON	ON
0011	1110	OFF	OFF

0 is even, 1 is odd

4). Discussion Questions

a. We initially divided the work as each one of us did one function, and then later we would decide who completed the other function based on the amount of time it took us to complete our initial functions. These functions we did were assigned as Ben did the adder, Evan did the subtractor, and Kimberly did the comparator. After completing our initial functions, we decided that Ben would primarily complete the top level file as he was the best at coding in VHDL, while Evan and Kimberly would complete the parity checker and tables in part 3 and the discussion questions in part 4. Also, Evan and Kimberly would assist with whatever coding needed to be done after they had finished the tables and questions.
b. Behavioral VHDL is what we use to program the digital logic for a function using typical programming statements to define what happens with inputs and outputs. It allows us to use combinations of logic gates to create different kinds of boolean functions.
c. Structural VHDL is used to create our top level schematic to let us utilize the functions created with behavioral VHDL and then choose which function to demonstrate on the board.
d. When trying to run the code for the parity bit, the output was not coming out to be expected, which we later realized was due to a syntax error in the port declaration. Overall, the only thing we found particularly difficult was coding in VHDL and implementing them into one cohesive design.
e. n/a